AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_vhdl2_individuell.pdf Version : V1.0	

TEIS ECS - Embedded Computer System -

Darwin Rengifo

2019-02-19

Sammanfattning:
Igenom den rapport ska vi se hur fungerar dubbelportsminnet på FPGA kretsen och deras implementering på VHDL koden.

Innehåll

1. INLEDNING 3
2. VAD ÄR DUBBELPORT RAM-MINNE? 3
3. RTL BESKRIVNING 3
4. BESKRIVNING AV TRUE-DUAL PORT RAM (DUAL KLOCK) 4
5. EXEMPEL AV SKRIVCYKEL OCH LÄSCYKEL 6
6. DO FILEN 7
7. TEST BENCH FILEN 7
8. SIMULERING AV DUBBELPORTSMINNET 9
9. SER UT PÅ CHIPET 11
10. REFERENSER 14

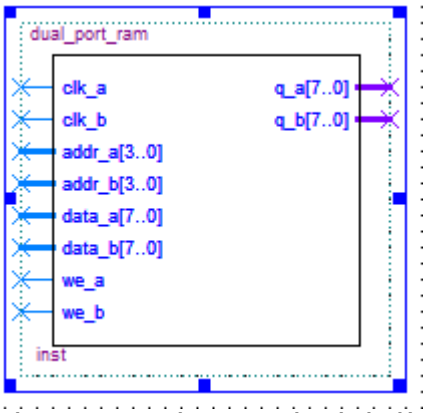
1. INLEDNING

De Denna rapport beskriver konfiguration metoden för ett dubbelport RAM-minne (dual klock) med mallarna som finns i Quartus.

2. VAD ÄR DUBBELPORT RAM-MINNE?

Ett dubbelport RAM-minne är delade minnen som kan skriva och läsa samtidigt i olika block av minne, detta minne har två(klockor, bussadress, databussen, läs/skriv avkoda och databussen utgången).

På figur 1 visar portarna som använder på True Dual-Port Ram (Dual Klock).



Figur 1. True Dual-Port Ram (Dual Klock)

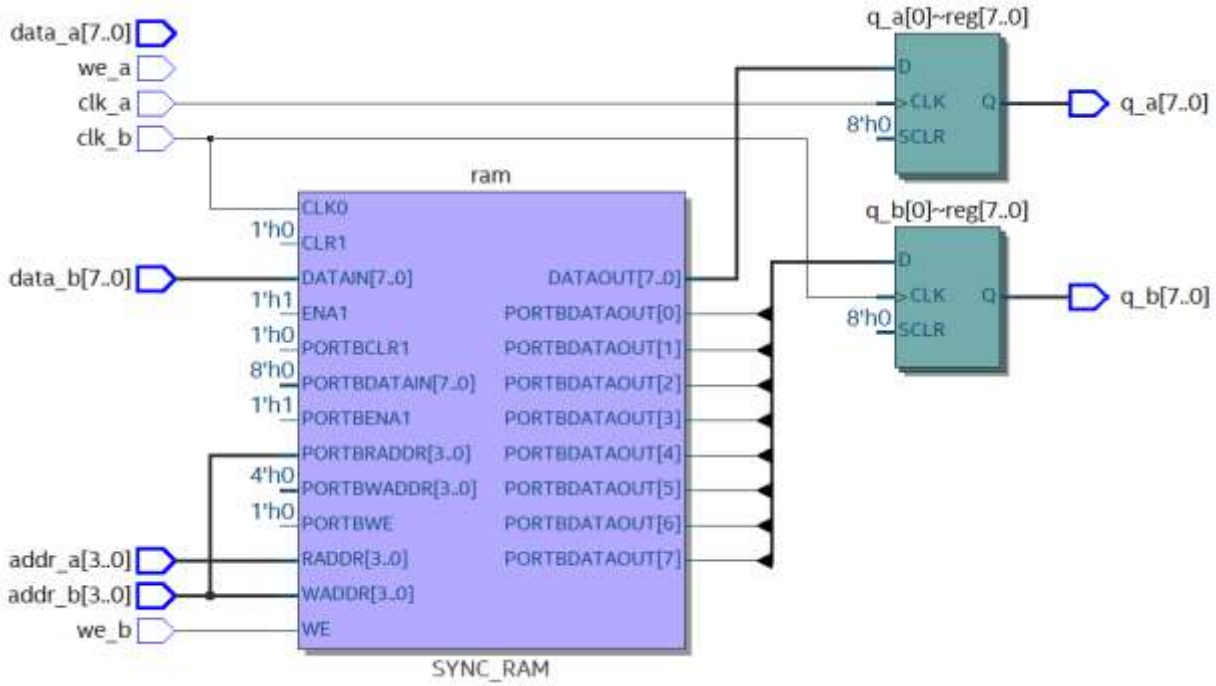
3. RTL BESKRIVNING

Figur 2 visar RTL. På den systemarkitektur visar hur clk_a(läsning) hanterar datautgången (DATAOUT[7..0]) genom port_a(q_a). Clk_b hanterar skrivningen (PORTBDATAOUT[0..7]) genom port_b och läser utgången genom utgången(q_b) av samma port_b.

Addr_a är minnesadressen som kommer att läsa genom port_a och adressen innehåll kommer att visa genom utgången(q_a).

Addr_b är minneadressen av port_b, där dataingången (DATAIN[7..0]) kommer att skrivas, minnet kommer att läsa denna adress innehåll genom port_b och kommer det att visar datautgången genom utgången(q_b).

Data_b är dataingången (DATAIN[7..0]) som kommer att skriver på minneadressen genom port_b.



Figur 2. RTL av dubbelportsminnet. En skrivcykel och läscykel

4. BESKRIVNING AV TRUE-DUAL PORT RAM (DUAL KLOCK)

På tabell 1 visar en dubbelport RAM-minne med duplicerade portarna för att kan använda på operationer av läser/skriver, två klockor för att jobbar med olika klockcykel. Designen enheten växlar dynamiskt mellan läs- och skrivoperationer med skriv aktivering ingång av respektive port.

En port är styrs för att avsändaren, som placerar data i minnet som databitar genom en skriv klocka.
Den andra porten styrs av mottagaren, som extraherar data från minnet.

Tabell 1. Dual Port Ram (Dual Klock) beskrivning

Port Name	Typ	Beskrivning
clk_a, clk_b	Input	Klock input
addr_a[16:0], addr_b[16:0]	Input	17-bit adress inputs av port A och port B
dataa[7:0], datab[7:0]	Input	8-bit data inputs av port A och port B

we_a, we_b	Input	Write enable inputs av port A och port B
q_a[7:0], q_b[7:0]	Output	8-bit data outputs of port A och port B

Den True Dual-Port Ram erkänner flera typer av kombinationer av två-porter: Minnet kan ha stödet av läs-/skrivning på båda två porten eller kan ha en skrivning port eller kan ha en läsning port på två olika frekvens klockan.

På det True Dual-Port Ram, kan du när som helst komma åt till alla platser i minnet från port A eller port B.

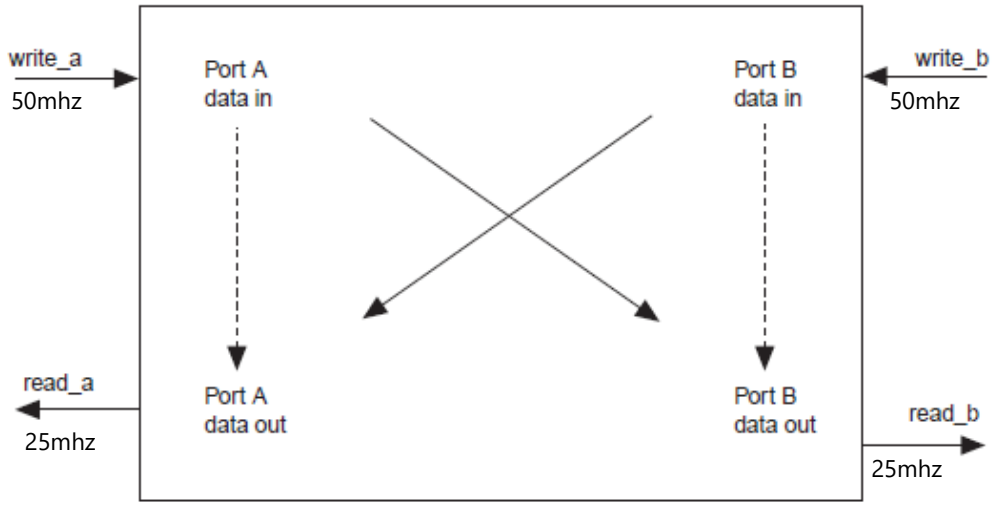
Dock, när den tillgången till samma minnesplats från båda portarna, bör du undvika eventuella skrivkonflikter.

När det försöker skriver på samma adressen plats från båda portarna samtidigt det händer en skrivkonflikt. Detta betyder att okända data lagras på den adressen.

På figur 3 visa funktionaliteten av dual-port minnet med dom olika kombinationer:

1. **Läs-/skrivning på båda två(samma-port dataflöde)**
   write_a → Port_A (data_in) → Port_A (data_out) → read_a
   write_b → Port_B (data_in) → Port_B (data_out) → read_b

2. **Skrivning på en port Läsning på annan port / (blandat-port dataflöde)**
   write_a → Port_A(data_in) → Port_b (data_out) → read_b
   write_b → Port_b(data_in) → Port_a (data_out) → read_a



Figur 3. True Dual-Port Ram (Dual Klock)

5. EXEMPEL AV SKRIVCYKEL OCH LÄSCYKEL

På figur 4 visa en exempel av en skrivcykel och läscykel av en dubbelportsminnet.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity dual_port_ram is

   port
   (
      clk_a    : in std_logic;
      clk_b    : in std_logic;
      addr_a   : in std_logic_vector(3 downto 0);
      addr_b   : in std_logic_vector(3 downto 0);
      data_a   : in std_logic_vector(7 downto 0);
      data_b   : in std_logic_vector(7 downto 0);
      we_a     : in std_logic;
      we_b     : in std_logic;
      q_a      : out std_logic_vector(7 downto 0);
      q_b      : out std_logic_vector(7 downto 0)
   );

end dual_port_ram;

architecture rtl of dual_port_ram is

   -- Build a 2-D array type for the RAM
   subtype word_t is std_logic_vector(7 downto 0);
   type memory_t is array(15 downto 0) of word_t;

   -- Declare the RAM
   shared variable ram : memory_t;

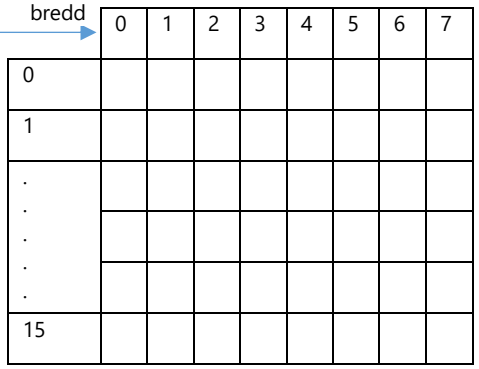
begin

-- Port A. LÄSCYKEL
   process(clk_a)
   begin
   if(rising_edge(clk_a)) then
      q_a <= ram(to_integer(unsigned(addr_a)));
   end if;
   end process;

   -- Port B. SKRIVCYKEL
   process(clk_b)
   begin
   if(rising_edge(clk_b)) then
      if(we_b = '1') then
         ram((to_integer(unsigned(addr_b)))) := data_b;
      end if;
      q_b <= ram(to_integer(unsigned(addr_b)));
   end if;
   end process;

end rtl;
```

dual_port_ram Entity



Figur 4. En skrivcykel och läscykel av dubbelportsminnet.

- **Port A. LÄSCYKEL**
  Ingång klock = varje 20ns ändrar flanken
  Genom den här process port_a kommer att utföra läsningen av adresserna innehåll som har skrivit genom port_b av minnet. På det här fallet använder inte det signalen we_a eftersom genom port_a kommer att läsa alla adresserna av minnet för att skriver data utgångarna(q_a) genom samma port_a.

- **Port B. SKRIVCYKEL**
  Ingång klock = varje 10ns ändrar flanken
  På denna process kan se skrivningen på minnet genom port_b när we_b är aktiv med '1', efter kommer att läsa denna adress av minnet för att skriver utgången(q_b) på samma port_b.

6. DO FILEN

DO filen innehålla extra signaler som kommer att visar på wave fönstret, en extra signal är RAM-minne, som visar hela RAM-minne med adresserna och data.

```
transcript on
if {[file exists rtl_work]} {
                    vdel -lib rtl_work -all
}
vlib rtl_work
vmap work rtl_work

vcom -93 -work work
{C:/Users/DARWIN/Pictures/Darwin/Estudios/TEIS/ELCURSO/VHDL_system_curs/(V6)_Power/dual_port_ram/dual_port_ram.vhd}

vcom -93 -work work
{C:/Users/DARWIN/Pictures/Darwin/Estudios/TEIS/ELCURSO/VHDL_system_curs/(V6)_Power/dual_port_ram/simulation/modelsim/dual_port_ram.vht}

vsim -t 1ps -L altera -L lpm -L sgate -L altera_mf -L altera_lnsim -L fiftyfivenm -L rtl_work -L work -voptargs="+acc"
dual_port_ram_vhd_tst

add wave *

add wave  \
sim:/dual_port_ram_vhd_tst/i1/ram

view structure
view signals
run 4 us
```

7. TEST BENCH FILEN

På figur 5 visar test bench filen som innehålla ett exempel av hur skriver i PORT B och läsa i PORT A av RAM-minne.

Den exempel jobbar med en klock period av 50ns för att kontrollerar skrivningen och läsningen processen.

Clk_b är klockan som kommer att kontrollerar skrivning processen i minnet. Den har en intervall av 20ns för varje ändring av positiv eller negativ flank, resultatet det kommer att blir 40ns för varje klockcykel.

Clk_b är klockan som kommer att kontrollerar läsning processen i minnet. Den har en intervall av 20ns för varje ändring av positiv eller negativ flank, resultatet det kommer att blir 40ns för varje klockcykel.

På den test_ram process det kommer att sätta först alla signaler med en initial värde för att undvika okända data.

- **SKRIVCYKEL**

  För att fylla på hela ram minne det finns en loop av 0-15 så det blir 16 adresser i hela minnet. För att startat skrivning på minnet det kommer att aktiverar signalen we_b(write enable) med '1' och så på den här sätt lämna minnet redo för att börjar skrivningen.

  Adresserna från 0-7 det kommer att skriver först, den variabeln "i" kommer att vara den räknare som kommer att innehålla adresserna som kommer att lagras i minnet. Som nästa steg det kommer att skriver data "11110000"(heltal-240) på varje adress som lagras i minnet.

  Det funkar på samma sätt när skriver från adresserna 10-15 men på det här fallet skriver data "00001111"(heltal-15) på varje adress som lagras i minnet.

  På fallet av adresserna 8 och 9, det kommer att skriver "0"(noll) på varje adress som lagras i minnet.

  Det finns två klocka perioder för varje skrivning i minnet, så det blir 100ns för varje skrivcykel.

  På figur 6 visar skrivcykel process.

- **LÄSCYKEL**

  För att läsa minnet det finns en loop av 0-15 så det blir 16 adresser som kommer att läsa i hela minnet.

  wait until rising_edge(clk_a), den betyder att väntar positiv flank av clk_a.

  Den variabeln "i" kommer att vara den räknare som kommer att innehålla adresserna från 0-15, som kommer att läsa från minnet.

  Det finns två klocka perioder för varje läsning av minnet, så det blir 120ns för varje läscykel.

  På figur 7 visar läscykel process.

```
-- Process för att skapa pulsklockor.
clk_a     <= NOT clk_a after 20 ns;   --25 MHz -- varje 20 ns byta värde
clk_b     <= NOT clk_b after 10 ns;   --50 MHz -- varje 10 ns byta värde

--- test__ram visar en test av skriver/läser av dual_port_ram minne
test_ram : process
begin
   --- initierar signaler
   we_a      <= '0';
   addr_a    <= (others=>'0');
   data_a    <= (others=>'0');
   we_b      <= '0';
   addr_b    <= (others=>'0');
   data_b    <= (others=>'0');
   wait for sys_clk_period*3;

   -- SKRIVCYKEL - PORT_B
   -- Det kommer att skriver data genom port_b av minnet.
   for i in 0 to 15 loop   -- skrivnig loop

      -- aktiverar skrivningen
      we_b      <= '1';

      if (i>=0 and i<=7) then -- från 0 till 7 skriver värde 240
         addr_b    <= std_logic_vector(to_unsigned(i, 4));
         data_b    <= "11110000"; --- 240

      elsif(i>=10 and i<=15) then  -- från 10 till 15 skriver värde 15
         addr_b    <= std_logic_vector(to_unsigned(i, 4));
         data_b    <= "00001111"; --- 15

      else
         addr_b    <= std_logic_vector(to_unsigned(i, 4)); -- från 8 till 9 skriver värde 0
         data_b    <= (others=>'0'); --- 0
      end if;
      wait for sys_clk_period*2;      -- Väntar klockperiod för nästa skrivning

   end loop;

   -- LÄSCYKEL - PORT_A
   -- Det kommer att läsa adresserna genom port_a av minnet.
   for i in 0 to 15 loop   -- läsning lopp

      wait until rising_edge(clk_a); -- väntar till nästa positiv flank
      addr_a    <= std_logic_vector(to_unsigned(i, 4));
      wait for sys_clk_period*2; -- Väntar klockperiod för nästa läsning

   end loop;

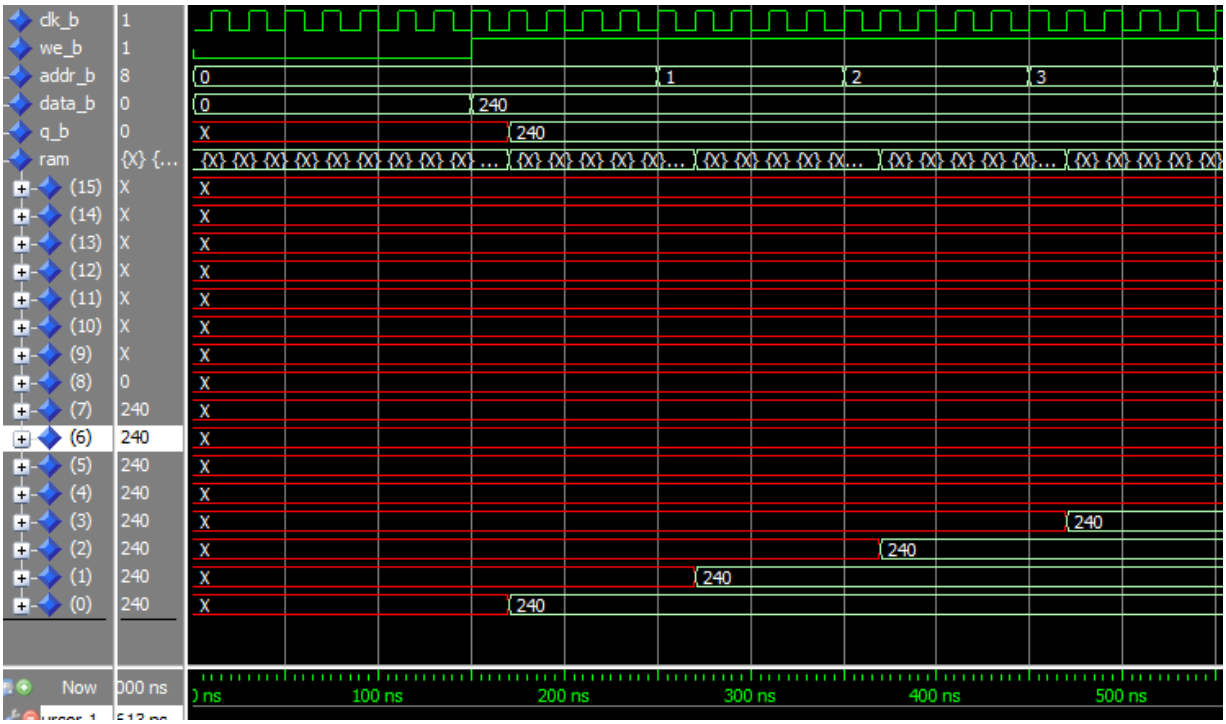
wait;
end process test_ram;
```

Figur 5. Test bench av dubbelportsminnet

8. SIMULERING AV DUBBELPORTSMINNET

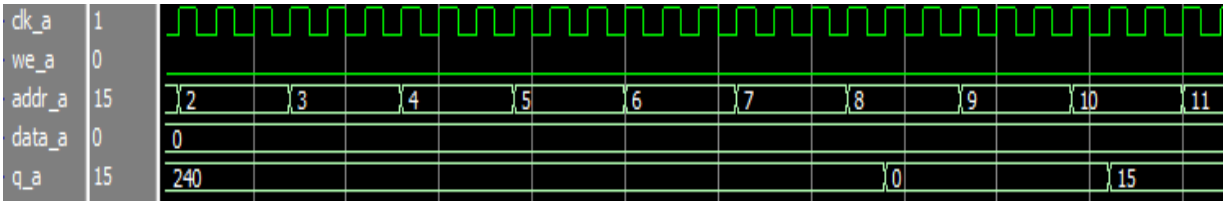
På simulering visar tre olika figurer, figur 6 visar en del av skrivcykel, figur 7 visar en del av läscykel och figur 8 visar hel simulering av båda två processer.

Skrivcykel: Den figur 6 visar skrivcykeln av adresserna och data i minnet. Dataläsningen är genom utgången signal "q_b", hela processen gör genom port_b.

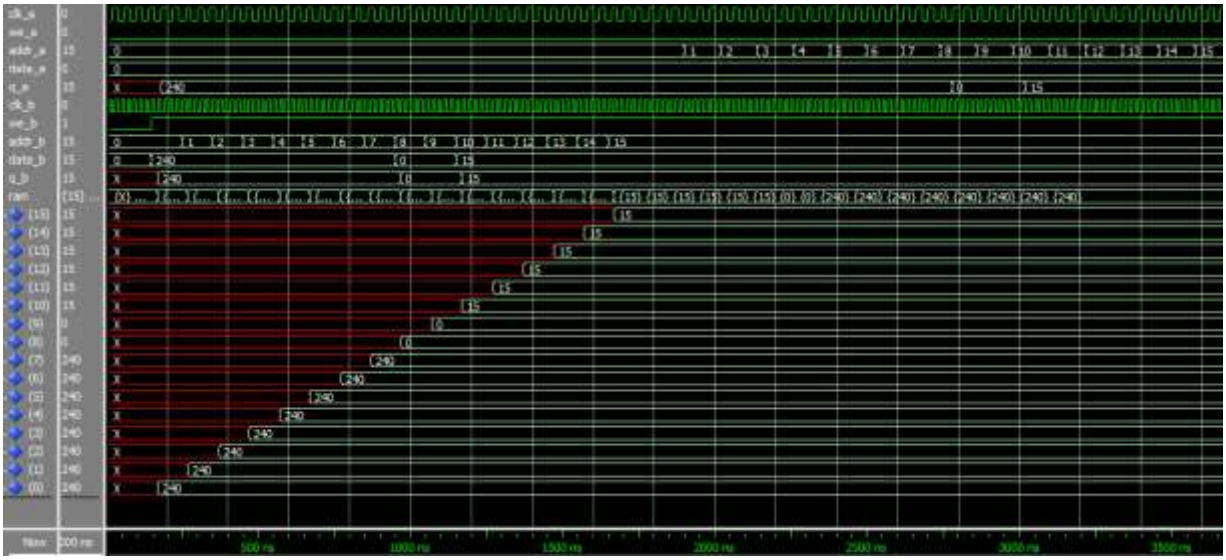


Figur 6. Simulering av dubbelportsminnet skrivcykel

Läscykel: Den figur 7 visar läscykeln av data som är innehålla i adresserna av minnet, data utgången är genom signalen "q_a", hela processen gör genom port_a.



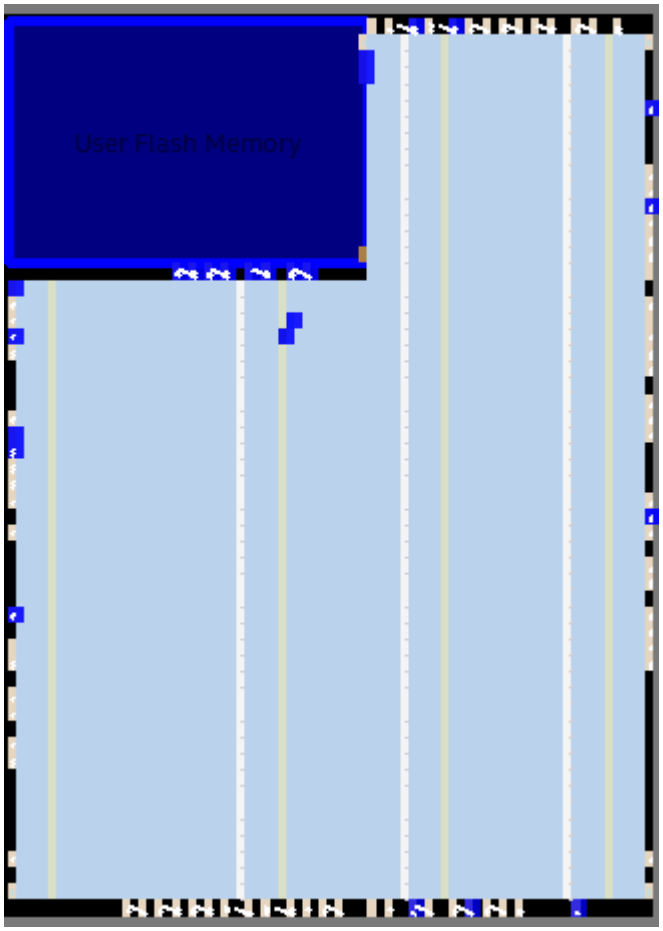
Figur 7. Simulering av dubbelportsminnet läscykel



Figur 8. Simulering av dubbelportsminnet läs-/skriv cykel

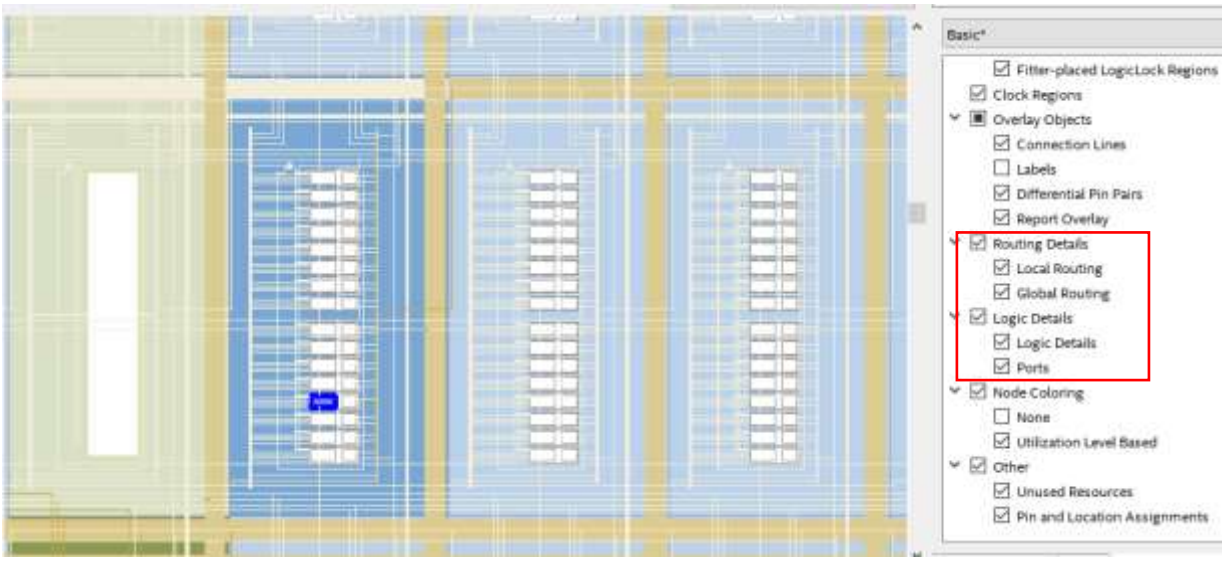
9. SER UT PÅ CHIPET

Det logiska blocken i designen är representerade i en blå färg och den är som indikerar hur det dubbelportsminnet är utvecklas på chipet. Ser figur 9.



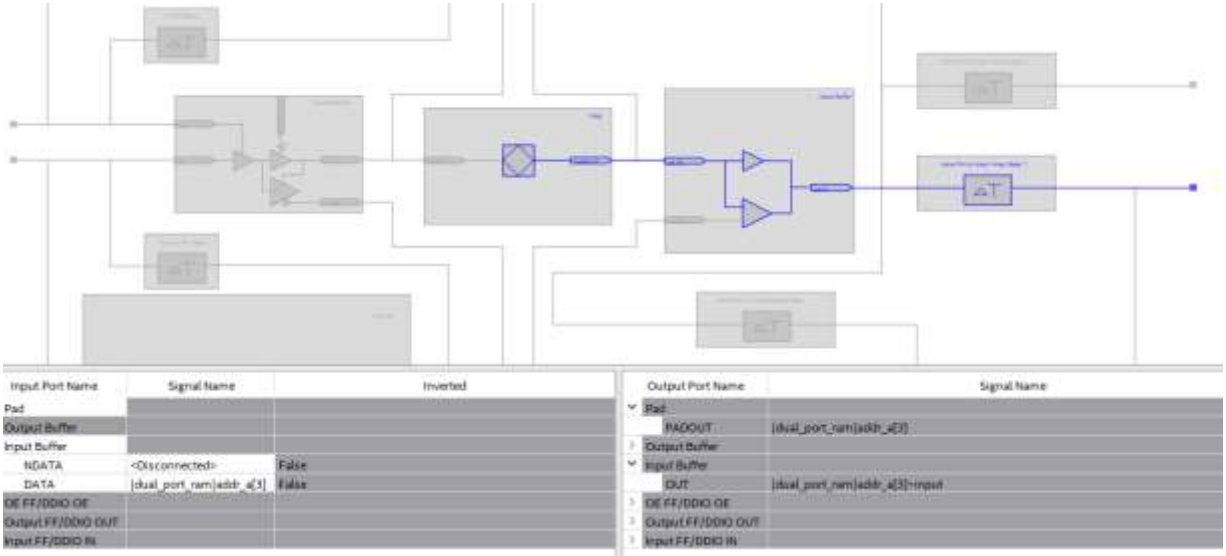
Figur 9. Dubbelportsminnet chipet

För att få en bättre syn på enhetens inre, aktiverar Routing Details – Local / Global)" och "Logic Details - Ports". Local Routing är vit linjer som är kopplat till logiska blocken och Global Routing är beige linje som är kopplat till alla ingången och utgången av minne banken. Ser figur 10.



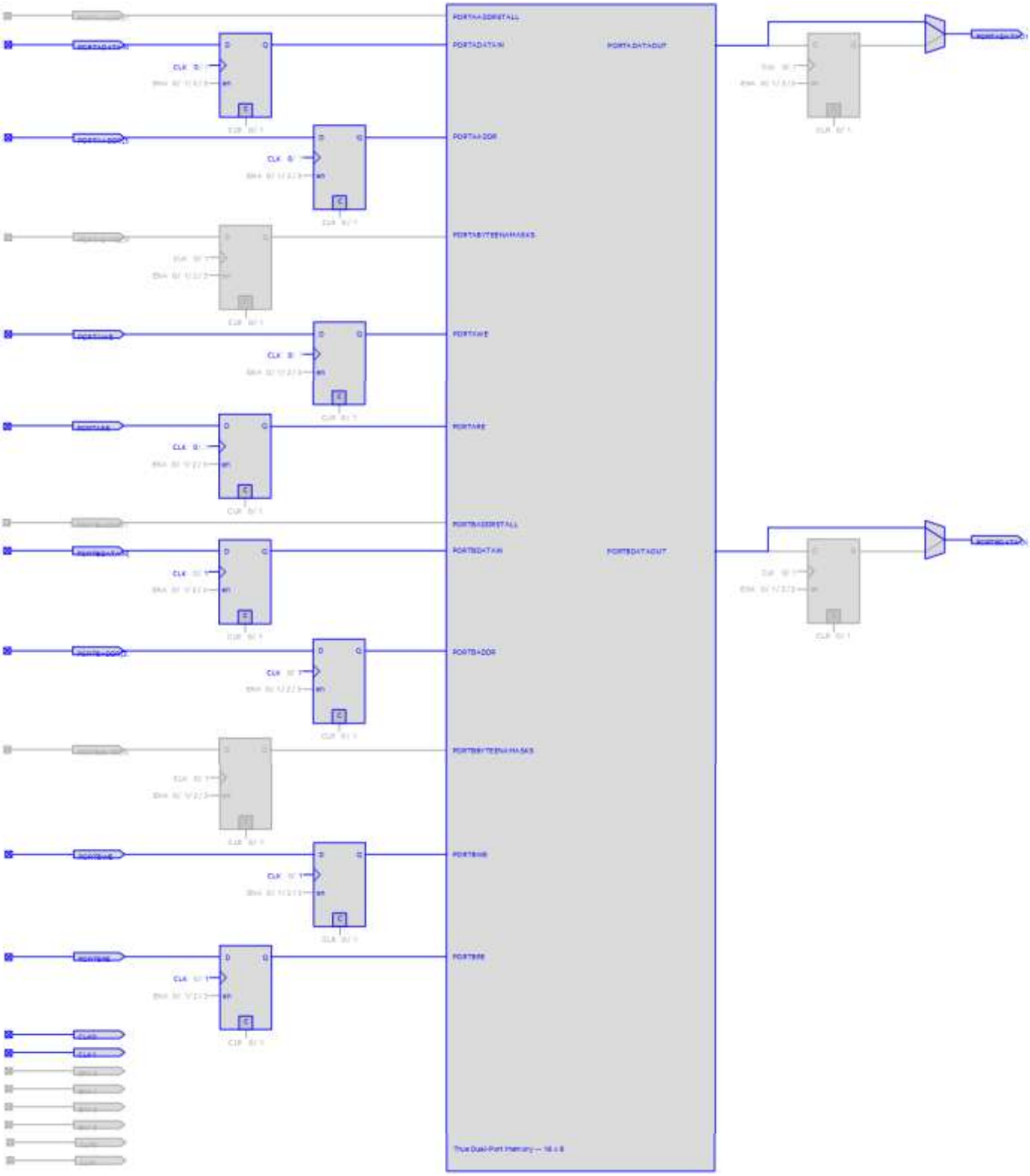
Figur 10. Routing och Logic Details av dubbelportsminnet chipet

För att se detaljer av en specifik logiska blocken använd "Zoom" verktyget och double-klick på det för att se logiska design på chipet. På figur 11, det finns tre logiska elementen som visar ingången och utgången processen av en läsadress genom port_a.



Figur 11. Logiska blocken av dubbelportsminnet chipet

På figur 12 ser ut RTL designen av dubbelportsminnet på chipet och alla ingången och utgången portar.



Figur 12. RTL på chipet

På figur 12 ser ut alla portar som är kopplat och utkopplat RTL designen av dubbelportsminnet på chipet.

Input Port Name	Signal Name	Inverted
PORTADATAIN[7..0]	<Multiple Signals>	False
PORTAADDR[3..0]	<Multiple Signals>	False
PORTABYTEENAMASKS	<Disconnected>	False
PORTAWE	|dual_port_ram|we_b~input	False
PORTBDATAIN[7..0]	|dual_port_ram|~GND	False
PORTBADDR[3..0]	<Multiple Signals>	False
CLK0	|dual_port_ram|clk_b~inputclkctrl	False
CLK1	|dual_port_ram|clk_a~inputclkctrl	False
ENA0	<Disconnected>	False
PORTAADDRSTALL	<Disconnected>	False
PORTBADDRSTALL	<Disconnected>	False
ENA1	<Disconnected>	False
ENA2	<Disconnected>	False
ENA3	<Disconnected>	False
CLR0	<Disconnected>	False
CLR1	<Disconnected>	False
PORTBBYTEENAMASKS	<Disconnected>	False
PORTBWE	GND	True
PORTARE	VCC	False
PORTBRE	VCC	False

Output Port Name	Signal Name
PORTADATAOUT[7..0]	<Multiple Signals>
[0]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a0
[1]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a1
[2]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a2
[3]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a3
[4]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a4
[5]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a5
[6]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a6
[7]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a7
PORTBDATAOUT[7..0]	<Multiple Signals>
[0]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a0~PORTBDATAOUT0
[1]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a1~PORTBDATAOUT0
[2]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a2~PORTBDATAOUT0
[3]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a3~PORTBDATAOUT0
[4]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a4~PORTBDATAOUT0
[5]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a5~PORTBDATAOUT0
[6]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a6~PORTBDATAOUT0
[7]	|dual_port_ram|altsyncram:ram_rtl_0|altsyncram_9pl1:auto_generated|ram_block1a7~PORTBDATAOUT0

Figur 13. Portarna på chipet

10. REFERENSER

[1] Memory Blocks in the Cyclone III Device Family - True Dual-Port Mode
https://www.intel.com/content/dam/www/programmable/us/en/pdfs/literature/hb/cyc3/cyc3_ciii51004.pdf

[2] Recommended HDL CodingStyles - True Dual-Port Synchronous RAM
https://people.ece.cornell.edu/land/courses/ece5760/DE1_SOC/HDL_style_qts_qii51007.pdf

[3] Quartus: Chip Planner - https://www.youtube.com/watch?v=xOS55lxv3AE